

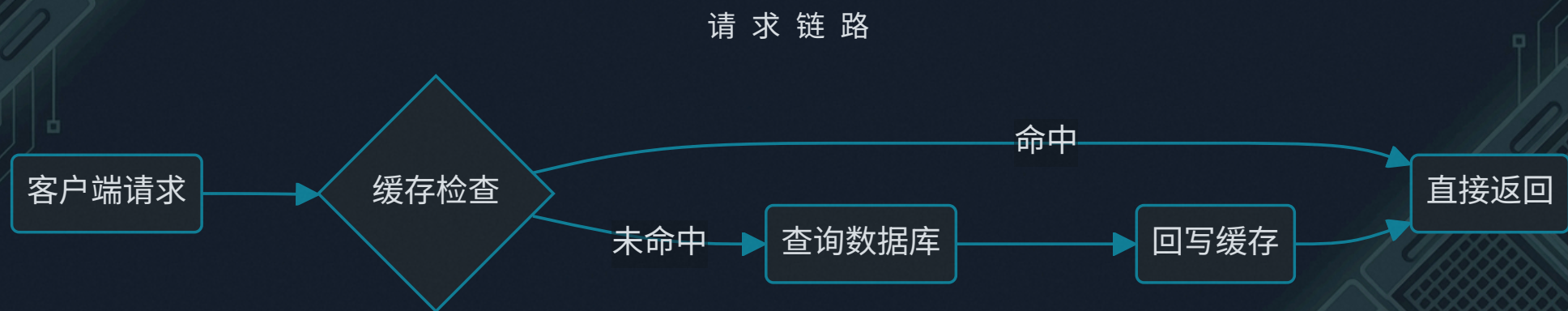
Redis 缓存穿透、击穿与雪崩

面向后端工程师的 Redis 缓存失效模式全解析



缓存的基本模型与三种失效

缓存的核心目的是保护数据库，但当缓存失效时，请求会直接压向数据库——而失效有三种截然不同的方式。



缓存穿透

查询不存在的数据，缓存永远不命中，每次请求直达数据库

缓存击穿

热点 Key 过期瞬间，大量并发请求同时查数据库

缓存雪崩

大量 Key 同时过期或 Redis 宕机，请求全面涌入数据库

缓存穿透：查询不存在的数据

请求的数据在缓存和数据库中都不存在，缓存永远不命中，每次请求都直达数据库。典型场景：恶意攻击用不存在的 ID 大量请求，或业务逻辑错误查询无效数据。

穿透过程



缓存空值

数据库查不到时，把空结果也缓存起来，设置较短 TTL（2-5 分钟），防止内存膨胀

布隆过滤器

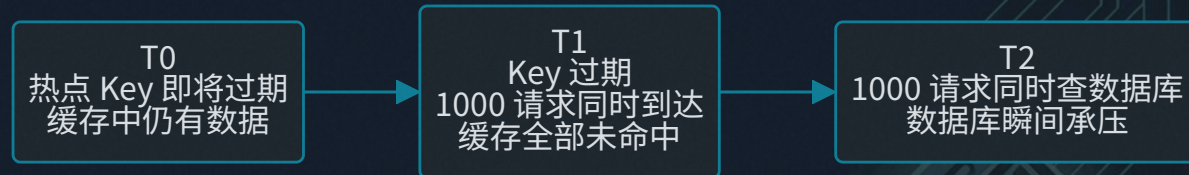
将所有合法 Key 预加载到布隆过滤器，请求先过过滤器。说"不存在"一定准确，说"存在"可能误判

缓存击穿：热点 Key 过期的瞬间

某个被高频访问的**热点 Key**在过期瞬间，大量并发请求同时发现缓存未命中，同时去查数据库。

与穿透的区别：数据本身存在，只是过期那一刻出现并发冲击。

击穿过程



互斥锁 保安全

只允许一个线程查数据库并重建缓存 `SET NX EX`，其他线程等待重试。牺牲短暂延迟保数据库安全。

逻辑过期 保可用

Key 不设实际 TTL，值中存逻辑过期时间。发现过期后当前线程返回旧数据，异步线程重建缓存。

缓存雪崩：大量 Key 同时失效

大量缓存 Key 在同一时间过期，或 Redis 实例宕机，导致请求全面涌入数据库。

与击穿的区别：击穿是单个热点 Key，雪崩是大面积 Key 同时失效。

雪崩过程

T0 时刻
10000 个 Key
设置相同 TTL (30 分钟)

T1 时刻
30 分钟后
10000 个 Key 同时过期

T2 时刻
请求全面穿透缓存
数据库承压甚至宕机

随机 TTL

基础过期时间 + 0~300 秒随机偏移，
打散过期窗口，避免集中失效

多级缓存

本地缓存 (Caffeine/Guava) +
Redis, Redis 失效时本地缓存仍能拦
截部分请求

限流降级

超过阈值时返回默认值或降级数据
(Sentinel/Hystrix)，保系统不崩

三者对比：一张图分清三个问题

穿透、击穿、雪崩的触发条件、影响范围和数据库压力特征各不相同，一张对比表可以快速诊断当前面对的是哪种问题。

穿透

触发条件

数据不存在

影响范围

特定请求

数据库压力

持续低压穿透

核心解法

缓存空值 + 布隆过滤器

击穿

触发条件

热点 Key 过期

影响范围

单个热点数据

数据库压力

瞬间高压冲击

核心解法

互斥锁 / 逻辑过期

雪崩

触发条件

大量 Key 同时过期

影响范围

全局

数据库压力

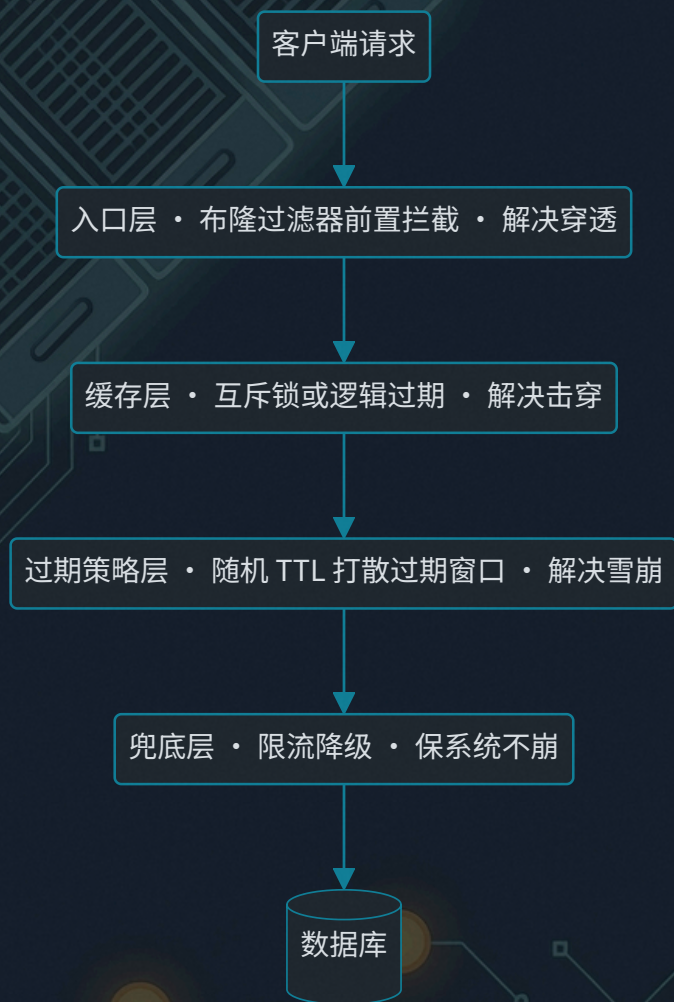
持续高压洪峰

核心解法

随机 TTL + 多级缓存 + 限流降级

生产环境：分层防御策略

在请求链路的每一层部署对应的防御手段，形成分层纵深防御体系



监控告警要点

缓存命中率

低于 90% 需排查穿透或过期策略异常

数据库连接池

持续高位说明缓存层未能有效拦截请求

热点 Key 监控

用 `redis-cli --hotkeys` 或 Redis LFU 策略识别热点

核心原则

穿透

防的是假请求

数据根本不存在，用缓存空值和布隆过滤器拦截无效查询

击穿

保的是热点 Key

过期瞬间的并发冲击，用互斥锁或逻辑过期串行化重建

雪崩

解的是集中失效

让过期时间分散、多级缓存分担、限流降级兜底

缓存设计的本质，是在数据库前面构建一个有弹性的保护层。

穿透、击穿、雪崩——三种问题，一个目标：保护数据库